

The Basics of Binary Exploitation

Binary exploitation works on the principle of turning a weakness into an advantage. In this article, the author deals with the basics of binary exploitation.



Binary exploitation involves taking advantage of a bug or vulnerability in order to cause unintended or unanticipated behaviour in the problem.

Basics required for binary exploitation

Binary exploitation might appear to be a strange topic but once you get started on it, you won't be able to stop. To get started, you need to know how the process memory is organised and how the stack is framed.

Processes are mainly divided into three regions: the text region, data region, and stack region.

The text region contains the data of the program and the executable file. You can only read the data and if you try to write the data, it will throw up a segment violation.

For easy understanding, the data segment is divided into three segments: data, BSS and heap. The data region contains global and static variables used in the program. The segment is further classified into two areas—for read-only data and the read-write area. The BSS segment has (uninitialised data) all global variables and static variables that are initialised to zero. Heap is usually managed by *malloc*, *free*, *realloc*, etc, where dynamic memory allocation is done

Stack is a type of abstract data type and it is LIFO (Last In First Out). It has a continuous block of memory containing data. The entire operations of the stack are controlled by the kernel. It is a continuous block of memory containing data in which the bottom of the